ERRORS IN KUBERNETS

CrashLoopBackOff

Conditions of Pod

1. Container starts
2. Application crashes
3. Container stops
4. Kubernetes restarts it
5. Application crashes again
6. CrashLoopBackOff

Check the Root Cause

1. kubectl get pods
2. kubectl describe pod <pod-name>
3. kubectl logs <pod-name>
4. kubectl logs <pod-name>
5. kubectl logs <pod-name> --previous

Common causes

- Application error
- Wrong environment variable
- Wrong database configuration
- Database unavailable
- Incorrect command/entrypoint
- Missing configuration
- Missing Secret
- Missing ConfigMap
- Application exits immediately

Example

1. If logs show
   Connection refused to database
2. The problem may be that your application cannot connect to MySQL/PostgreSQL.
3. Fix the database connection and redeploy.

ImagePullBackOff

Kubernetes cannot download the container image.

Check Root Cause
kubectl describe pod <pod-name>

Common Causes

1. Wrong image name
2. Wrong image tag
3. Private Docker registry
4. Authentication problem
5. Image doesn't exist

ErrImagePull

Check Root Cause

1. kubectl describe pod <pod-name>

Check whether the image actually exists:

2. Docker Hub / ECR / private registry

Pending

The Pod has been created but Kubernetes cannot schedule it onto a node.

Check Root Cause

1. kubectl describe pod <pod-name>

Common messages:

1. Insufficient cpu
2. Insufficient memory
3. No nodes available
4. node selector mismatch
5. taint/toleration problem
6. PVC not available

OOMKilled

Your container used more memory than allowed.

Check Root Cause

1. kubectl describe pod <pod-name>

You see:
Reason: OOMKilled

Example:

```
resources:
  limits:
    memory: "128Mi"
```

If the application requires more than 128 MiB, it can be killed.

Increase the limit:

```
resources:
  limits:
    memory: "512Mi"
```

Error / ContainerCreating

1. If you see
   ContainerCreating
2. For a long time

Check Root Cause

1. kubectl describe pod <pod-name>

Common causes:

1. Volume problem
2. Secret problem
3. ConfigMap problem
4. CNI/network problem
5. Image pulling
6. Storage problem

Service Not Accessible

1. Suppose pod is running
2. But the application not accessible

Check

1. kubectl get svc
2. kubectl describe svc <service-name>

common problem

service
selector:
  app: nginx

pod
labels:
  app: web

They don't match.

Node NotReady

Check:

1. Kubectl get nodes
2. kubectl describe node <node-name>

Possible reasons

1. kubelet problem
2. CNI problem
3. Disk pressure
4. Memory pressure
5. Network problem
6. Node instance problem

Pod is Running but Application Doesn't Work

Check

1. Kubectl get pods
2. Kubectl describe pod <pod-name>
3. Check
   { Liveness probe / Readiness Probe / Startup Probe }

Example:

Readiness probe failed

means Kubernetes considers the application not ready to receive traffic.

Pending PVC

If you see pvc is in pending state
PersistentVolumeClaim

Check

1. Kubectl get pvc
2. kubectl describe pvc <pvc-name>

Common Causes

1. No StorageClass
2. Incorrect StorageClass
3. No available PV
4. EBS/EFS configuration issue
5. CSI driver problem

Check

1. kubectl get storageclass

For EKS, also check your CSI driver if you're using EBS/EFS storage

---

CreateContainerError

Check

1. kubectl describe pod <pod-name>
2. kubectl logs <pod-name>

Common Causes

1. Incorrect command
2. Incorrect volume mount
3. Permission issue
4. Invalid container configuration

When a Kubernetes resource has an error:

1. GET
2. DESCRIBE
3. LOGS
4. EVENTS
5. FIX
6. VERIFY
